Language Models and Agents: Individual Project Report

Korlapati Keerthana
2023101121

1 Phase 1: Corpus Construction and Tokenization

1.1 Language Selection

The project requires one higher-resource non-English Indian language and one lower-resource language from the specified list. Telugu was selected as the higher-resource language and Nepali as the lower-resource language.

Telugu was chosen as the higher resource language after finding substantially more text through publicly available sources. And also beacuse I can read Telugu, so can evaluate better. Nepali was selected from the permitted lower-resource language list and was processed independently from Telugu throughout the project.

1.2 Corpus Construction

For both languages, the corpus pipeline collected source documents, removed invalid records and exact duplicates. Then all sources were merged, the data unicode normalized and deduplicated, and made into independent training, validation, and test sets. This entire corpus pipeline was done separately for both the languages.

Table 1: Corpus construction summary.

	Telugu	**Nepali**
Initial target	600M tokens	600M tokens
Final training corpus	486.03M tokens	500M tokens
Manual tokens available	831.0M	109.4M
Manual tokens selected	150.0M	109.4M
Manual fraction	25.00%	18.24%
Non-manual fraction	75.00%	81.76%
Validation tokens	2.92M	10M
Test tokens	2.82M	10M

The Telugu corpus contained approximately 4.01M documents. The collected material included Telugu news, Wikisource, Wikipedia, Internet Archive, Gutenberg, thesis material, and IndicCorp.

The Nepali corpus contained approximately 1.01M documents, with FineWeb2 and OnlineKhabar contributing majorly. Other sources included Nepali Wikipedia, OCR material, parliamentary proceedings, the Law Commission, and archival material.

The project requires at least 20% of the final training tokens to come from manual collection. Telugu had sufficient manual material, allowing 150M manually collected tokens to be used. This gives a 25% manual fraction.

For Nepali, only approximately 109.4M manually collected tokens were available. Consequently, 20% of a 600M-token corpus could not be reached using the available manual material. The selected corpus therefore contains approximately 109.4M manual tokens, corresponding to 18.24%.

1.3 Train/Test Leakage Prevention

The validation and test data were selected separately from the training data. For Telugu, training-record hashes were loaded before sampling validation and test records. The final verification showed zero overlap between training and validation, training and test, and validation and test.

For the synthetic reasoning data, the generator also checks for exact prompt overlap across splits. Train/validation templates and test templates use separate template-ID families, while entity overlap across splits is allowed. This is intentional because the task is supposed to test the reasoning relation rather than whether the model has seen a particular entity name before.

The generator also checks global prompt uniqueness and prevents test template leakage into train or validation.

1.4 Tokenizer Construction

A separate BPE tokenizer was trained from scratch for each language.

Table 2: Final tokenizer statistics on held-out validation text.

	Telugu	**Nepali**
Vocabulary size	6,000	8,000
Validation documents	10,000	10,000
Validation tokens	2,922,947	5,718,485
Fertility	2.2417	1.6672
Unknown tokens	0	0
Unknown-token rate	0%	0%
Tokens / character	0.2830	0.2578
Characters / token	3.53	3.88
Unique validation tokens	4,387	4,597
Unused vocabulary entries	1,613	3,403

For Telugu, vocabulary sizes from 5K to 10K were compared. Increasing the vocabulary reduced fertility, with fertility decreasing from 2.3850 at 5K to 1.9361 at 10K. The 6K vocabulary size was chosen as a compute-aware compromise between token fragmentation and vocabulary size.

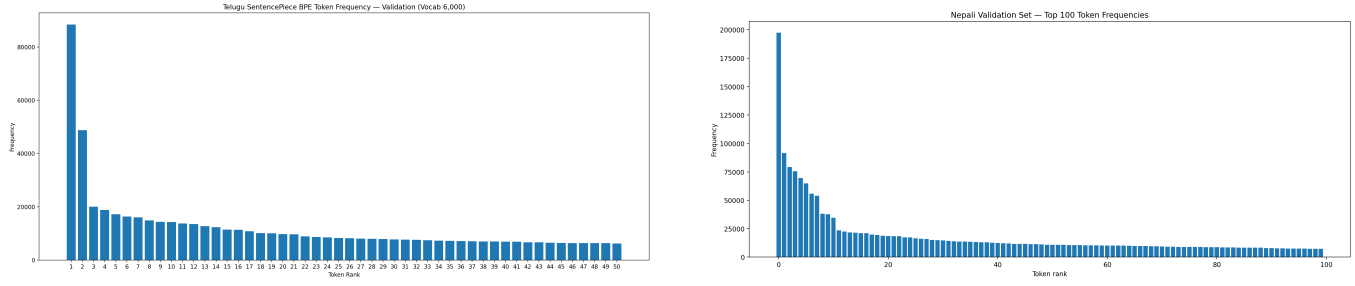
Table 3: Telugu tokenizer vocabulary sweep.

Vocabulary	5K	6K	7K	8K	9K	10K
Fertility	2.3850	2.2417	2.1358	2.0542	1.9911	1.9361
Tokens/char	.3011	.2830	.2697	.2594	.2514	.2444

The Nepali tokenizer uses an 8K vocabulary and obtains zero unknown tokens with fertility 1.6672. Large differences between sources were observed. The OCR material has fertility of approximately 6.22, compared to approximately 1.54 for OnlineKhabar and 1.67 for FineWeb2. This can be interpreted as OCR noise producing more fragmented token sequences.

1.5 Token Frequency Statistics

Token frequencies were also calculated on held-out validation text. The Telugu plot below shows the most frequent tokens for the final 6K tokenizer, while the Nepali plot shows the corresponding statistics for the final 8K tokenizer.



(a) Telugu BPE, 6K vocabulary

(b) Nepali BPE, 8K vocabulary

Figure 1: Top-token frequency plots on the validation sets.

The frequency distributions are concentrated around a relatively small number of frequent tokens, while a large part of the vocabulary is used less often. In the validation data, 4,387 of the 6,000 Telugu tokens and 4,597 of the 8,000 Nepali tokens were observed.

1.6 Tokenizer Examples

The final tokenizer notebooks were also used to inspect how example sentences are split into tokens and to check that decoding the token IDs gives back the original sentence.

Language	**Sentence → tokenized output**
Telugu	తెలుగు భారతదేశంలో మాట్లాడే ఒక ప్రధాన భాష. → '▁తెలుగు', '▁భారతదేశ', 'ంలో', '▁మాట్లా', 'డే', '▁ఒక', '▁ప్రధాన', '▁భాష', '.'
Nepali	नेपाल प्राकृतिक सुन्दरताले भरिएको देश हो। → 'नेपाल', 'प्राकृतिक', 'सुन्दर', 'ताले', 'भरि', 'एको', 'देश', 'हो', '।'

Table 4: Tokenizer examples from the final tokenizer notebooks.

The tokenizer checks also confirmed that the selected final tokenizers produce zero unknown tokens on the held-out validation sets.

2 Phase 2: Model Implementation, Pretraining, and Evaluation

2.1 Model Configuration and Design Decisions

For Phase 2, the following architecture was selected after examining the tokenizer statistics and the approximately 25M-parameter constraint given.

Table 5: Final Transformer configurations.

	Model H: Telugu	**Model L: Nepali**
Vocabulary size	6,000	8,000
Model dimension	448	448
Transformer layers	10	11
Attention heads	8	8
FFN dimension	1,344	1,344
Context length	512	512
Dropout	0.10	0.10
Positional encoding	Learned absolute	Learned absolute
Normalization	Pre-norm	Pre-norm
Embedding tying	Yes	Yes
Trainable parameters	23,042,432	25,950,848

The same model dimension and number of attention heads were used for both languages so that the comparison is more controlled. The number of layers differs by one because the two models have different vocabulary sizes, which also changes the number of parameters used by the embedding and output layers.

Learned absolute positional embeddings were used because they give the model direct information about token order and were also simple to implement from scratch. Since the maximum sequence length is set to 512, the position table is also limited to 512 positions. This means the model has a fixed context length of 512 tokens.

Pre-norm Transformer blocks were used to make the training more stable when stacking multiple Transformer blocks. The normalization is applied before the attention and feed-forward sublayers.

The output embedding matrix is tied to the input embedding matrix. This saves some parameters and helps keep the models within the approximately 25M parameter limit.

2.2 Transformer Implementation

The models implement decoder-only causal self-attention directly using PyTorch primitives. For an input tensor

$$X \in R^{B\times T\times d},$$

queries, keys, and values are projected and reshaped from (B, T, d) to (B, h, T, d_k). Attention is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V,$$

where M is an upper-triangular causal mask.

The $1/\sqrt{d_k}$ factor keeps the query–key dot products from becoming too large as the head dimension increases. Without this scaling, the softmax can become too concentrated and make learning harder. The causal mask makes sure that position t cannot attend to positions after t.

Each Transformer block contains causal multi-head attention, a GELU feed-forward network, residual connections, layer normalization, and dropout.

2.3 Causal Mask Verification

The causal mask was checked empirically in addition to being present in the attention implementation.

For Telugu, the supplied verification changes only token position 6 in the input sequence. The model is run on the original and modified sequences in evaluation mode. Since position 6 is in the future for positions 0–5, the logits at positions 0–5 should not change. The test checks that the maximum absolute logit difference at all these positions is below 10^{-6}.

The same-position and later-position logits are allowed to change, since position 6 can see itself and position 7 can see position 6. The Telugu test passed.

For Nepali, the supplied causal-mask test directly checks the attention matrix and verifies that every future-token attention weight is exactly zero. The test also passed.

Thus, the implementation was checked both through the attention weights and, for Telugu, through the effect of changing a future input token on earlier logits.

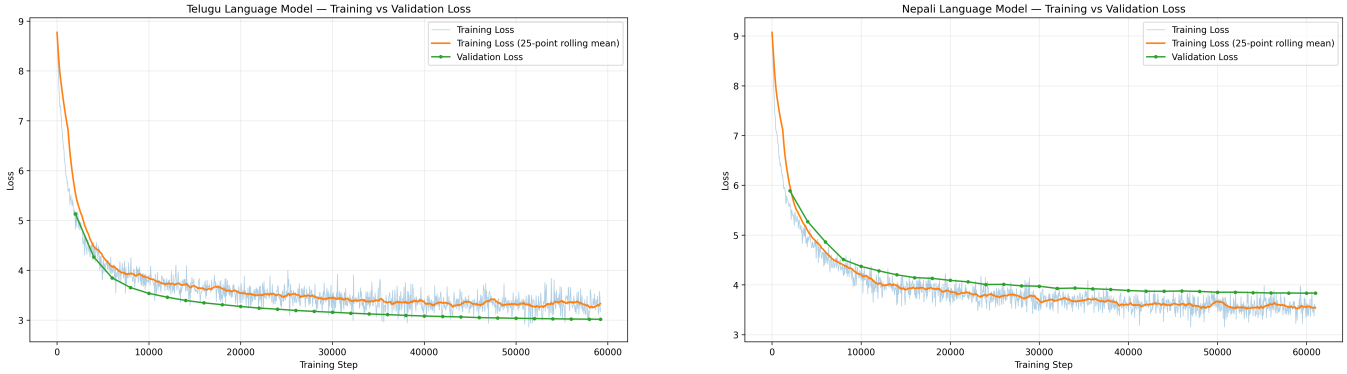
2.4 Pretraining

Both models were pretrained independently on their respective corpora. AdamW, learning-rate warmup, cosine learning-rate decay, validation evaluation, mixed precision, and checkpointing were used.

Table 6: Pretraining configuration.

	Telugu	**Nepali**
Training tokens	486.03M	500M
Validation tokens	2.92M	10M
Batch size	16	16
Peak learning rate	3×10^{-4}	2×10^{-4}
Training steps	59,214	61,000
Optimizer	AdamW	AdamW
Precision	FP16	FP16
GPU	Tesla T4	Tesla T4

The final validation losses were 3.016 for Telugu and 3.834 for Nepali. These correspond to validation perplexities of 20.40 and 46.25 respectively.



(a) Telugu (b) Nepali

Figure 2: Training and validation loss curves for the two models.

2.5 Language Modeling Evaluation

Table 7: Held-out language-modeling results.

Metric	**Model H: Telugu**	**Model L: Nepali**
Test cross-entropy	3.124	3.409
Test perplexity	22.73	30.23
Test BPB	0.467	0.846
Evaluated target tokens	2,824,704	9,999,872
Test UTF-8 bytes	27.28M	58.15M

Telugu obtained lower cross-entropy, perplexity, and BPB. The BPB difference is useful here because perplexity depends on the tokenization scheme. The lower Telugu BPB suggests that the difference is not only because the two models use different vocabulary sizes.

The results also match some of the differences seen during Phase 1. Both models were trained at approximately the same token scale, but Telugu had more available manually collected material and used a 25% manual selection. Nepali used the maximum available manual material but reached only 18.24%. Nepali also had an OCR component with much higher tokenizer fertility.

2.6 Generation Quality and Diversity

Generation was evaluated from held-out prefixes using greedy decoding and temperatures 0.5, 1.0, and 1.5.

Table 8: Generation quality and diversity. BLEU is corpus BLEU-4; chrF is chrF++; repetition is repeated 3-gram fraction.

Model	Setting	BLEU	chrF	ROUGE-L	Rep.	D-1	D-2
Telugu	Greedy	8.14	22.28	.137	.543	.302	.405
	$T = 0.5$	7.14	22.78	.127	.246	.424	.650
	$T = 1.0$	5.41	22.81	.097	.009	.727	.969
	$T = 1.5$	.40	17.09	.025	.001	.825	.997
Nepali	Greedy	3.43	16.66	.135	.712	.116	.226
	$T = 0.5$	3.42	18.61	.138	.390	.179	.460
	$T = 1.0$	1.62	18.92	.102	.014	.323	.894
	$T = 1.5$	.25	16.11	.058	.0002	.412	.981

Higher temperatures create more diverse results, while lower temperatures keep the output closer to the reference. There is a direct trade-off here. Increasing temperature reduces repeated n-grams and increases Distinct-1 and Distinct-2, but sufficiently high temperatures reduce BLEU and ROUGE-L.

For Telugu, repeated 3-grams fall from 54.3% under greedy decoding to 0.9% at $T = 1.0$, while Distinct-2 increases from 0.405 to 0.969. Nepali follows the same trend but starts from a more repetitive greedy baseline.

2.7 Actual Generated Samples

The following are actual outputs from the held-out generation runs. A single held-out prefix is shown for each language so that the change with temperature can be seen more easily.

	Telugu
Greedy	ఓడించి, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు పెట్టుకుని, ఆ పార్టీతో పొత్తు
$T = 0.5$	గట్టిగా నిలదీసింది. ఆ పార్టీతో పాటు, ఆ పార్టీతో కూడా విభేదాలు వచ్చాయి. ఆ పార్టీతో పాటు, ఆ పార్టీతోనూ విభేదాలు వచ్చాయి. ఆ పార్టీతోనూ విభేదాలు వచ్చాయి. ఆ పార్టీతోనూ విభేదాలు వచ్చాయి.
$T = 1.0$	ఆస్పత్రికి మార్చింది. అందువల్ల ఇక్కడ యాత్ర ముగించి అక్కడకు వెళ్ళి అక్కడి నుంచి జారిపోవాలని నిర్ణయించుకుంది. కాని ఆ రోజు తను పోలీసు బలగాలు చేతికి నిరసనగా వెళుతున్నప్పుడు కంట్రోల్ చేయడానికి కాని ఎవరి మీద మంత్రులు ఉన్నారు.
$T = 1.5$	జూన్ 16 టిడిపి ప్రతి రైడుతూ సలహాలు ఇవ్వాలనిపించింది. ఆలోచన విధాన ప నారాయణ, వంశీ, మీహి, సాయికృష్ణ ఈ మార్పు ముఖ్యం... సంజయమ్ము రవి, కళా ఆలోచనలు ఆయనకు ఉన్నాయి.
	Nepali
Greedy	मा रहेका मतदान केन्द्र हरुमा मतदान केन्द्र हरु निर्माण गरिएको छ । अ त्त रिया मा रहेका मतदान केन्द्र हरुमा मतदान केन्द्र हरु निर्माण गरिएको छ । अ त्त रिया मा रहेका मतदान केन्द्र हरुमा मतदान केन्द्र हरु निर्माण गरिएको छ ।
$T = 0.5$	मा रहेका मतदान केन्द्रमा मतदान अधिकृत र मतदान अधिकृत लाई मतदान अधिकृत ले मतदान अधिकृत ले मतदान अधिकृत लाई मतदान अधिकृत ले मतदान अधिकृत लाई मतदान अधिकृत ले मतदान अधिकृत ले मतदान अधिकृत लाई मतदान अधिकृत ले
$T = 1.0$	मा मात्र ३७ १ मतदाता रहेका छन् । अहिलेसम्म तिरेको ओझेलमा परेको उक्त मतदान केन्द्रमा मणि पाल, टेक बहादुर विकको कक्षा १० मा अध्ययनरत श्याम लाल चौधरी लगायतले मतदान गरेका छन् ।
$T = 1.5$	यता नेपाली खेल पत्रकार अभियानमा स्तरीय वाद तयार भएपनि तस् होस् : घनन्देही, ३१ साउन उजुरी रको अर्को फैसला वर्तमान विषय घटना भएको ठा दद काउन्सिलका बसोवाससँग मिल्दो द्वासाईकेट (बग एउटै भवन व्यवस्था पक्षले बनेको बाग बजार विकास

Table 9: Actual generated continuations for one held-out prefix at different temperatures.

Qualitatively, the greedy outputs in both languages are quite repetitive. At $T = 0.5$, some variation is introduced but repeated phrases are still common. At $T = 1.0$, the outputs are more varied and contain longer new continuations, although they are not always fully coherent. At $T = 1.5$, repetition is very low but the text becomes less consistent and sometimes switches to unrelated topics or phrasing. The outputs are generally in the correct script, but language correctness and coherence are not consistent at higher temperatures.

BLEU and ROUGE-L therefore need to be interpreted together with the qualitative samples and diversity measures. The character-level chrF metric is also useful because exact subword overlap is not required.

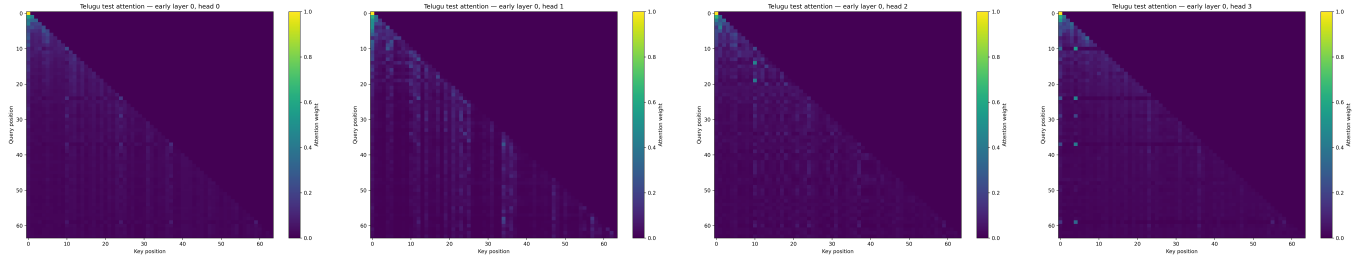
2.8 Phase 2 Attention Analysis

Attention was analyzed using heatmaps, entropy, and mean attention distance.

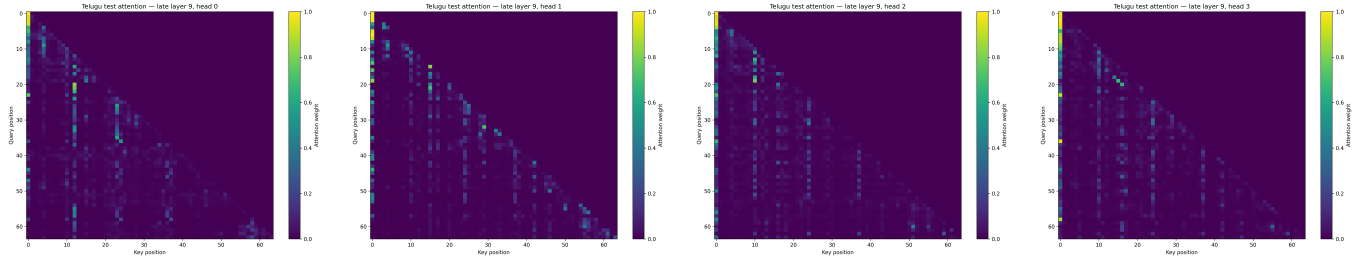
Table 10: Phase 2 attention statistics.

	Telugu	**Nepali**
Mean attention entropy	2.187	2.087
Mean attention distance	11.78	12.73
Early-layer distance	15.37	14.76
Late-layer distance	16.25	17.42

The following figures show multiple heads instead of only one representative head. Four heads from the early layer and four heads from the late layer are shown for each model.

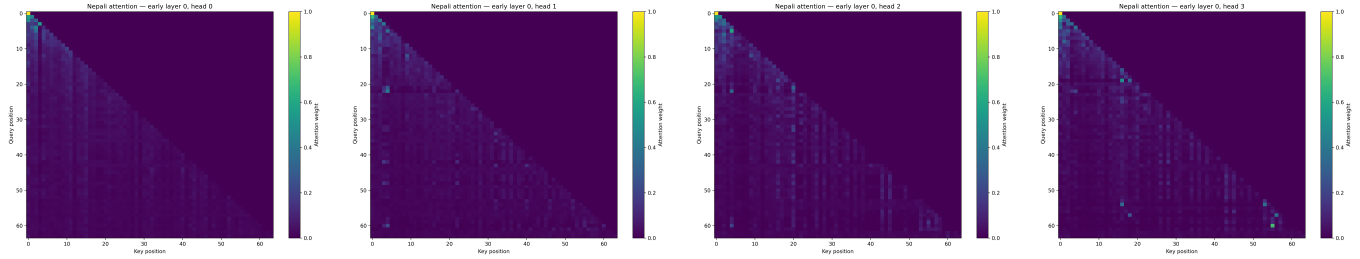


(a) L0 H0 (b) L0 H1 (c) L0 H2 (d) L0 H3

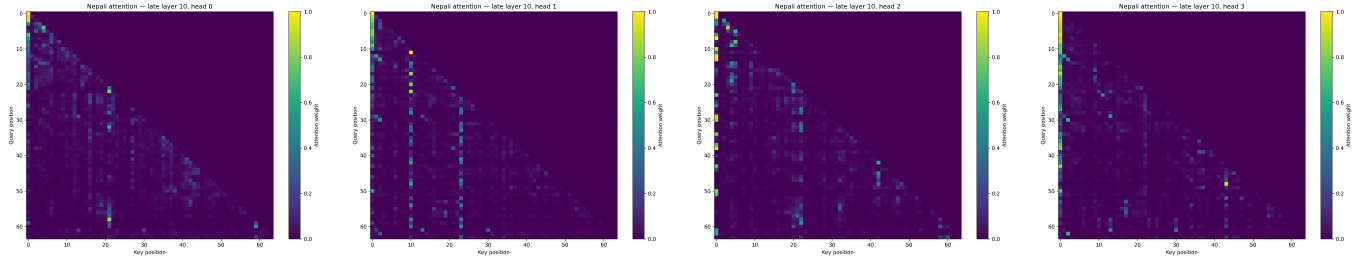


(e) L9 H0 (f) L9 H1 (g) L9 H2 (h) L9 H3

Figure 3: Telugu Phase 2 attention maps. The top row shows four heads from the early layer and the bottom row shows four heads from the late layer. Axes are query position and key position, with color representing attention weight.



(a) L0 H0 (b) L0 H1 (c) L0 H2 (d) L0 H3



(e) L10 H0 (f) L10 H1 (g) L10 H2 (h) L10 H3

Figure 4: Nepali Phase 2 attention maps. The top row shows four heads from the early layer and the bottom row shows four heads from the late layer.

The attention maps show that different heads behave differently. Some heads focus more on nearby positions, while others attend to positions farther away. Some heads also appear to attend based on the content of the tokens rather than only their position.

One interesting pattern is that the attention distance does not simply increase as the model gets deeper. Both models show broader attention in the early layers, more local attention in the middle layers, and broader attention again in the later layers. For Telugu, the mean distance goes from 15.37 in the first layer down to 4.34 around the middle layers, and then goes back up to 16.25 in the final layer. Nepali shows a similar pattern, going from 14.76 to 4.52 and then increasing to 17.42 in the final layer.

There are also clear examples of individual head specialization. For Telugu, Layer 5 Head 1 has very low entropy (0.097) and a mean distance of about 1.04, making it a strong local example. For Nepali, Layer 5 Head 5 has entropy

0.081 and mean distance 1.01. On the other side, Nepali Layer 7 Head 6 has a mean distance of 24.54, while Layer 7 Head 2 has a distance of 23.26. These show that the heads are not all using the context in the same way.

Overall, the two models show similar types of head specialization, although Nepali has a slightly higher overall mean attention distance (12.73 vs. 11.78) and a higher late-layer distance (17.42 vs. 16.25).

3 Phase 3: Reasoning Fine-tuning

3.1 Synthetic Reasoning Dataset

Each language uses a generated synthetic reasoning dataset. The main dataset contains 20,000 training, 2,000 validation, and 3,000 test examples.

The tasks include direct comparison, reverse comparison, equality, two-hop transitivity, three-hop transitivity, and ordering. Attributes include height, age, weight, price, and quantity.

Examples are generated independently for each language and use the target language's script and phrasing. Entity names, numerical values, relation directions, and statement ordering are varied to reduce template memorization.

The generator has explicit leakage checks. Exact prompts must be unique across train, validation, and test. Test templates are kept separate from train/validation templates. Entity names are allowed to occur in different splits, since the same reasoning task should still work with different entities. The original train, validation, and test files are not modified during the experiments.

3.2 Fine-tuning Results

The primary Phase 3 comparison uses the pure synthetic reasoning condition containing 20K reasoning examples and 0K original pretraining examples.

Table 11: Reasoning performance before and after fine-tuning.

	Telugu	**Nepali**
Test examples	3,000	3,000
Pretrained exact match	0.00%	0.00%
Fine-tuned exact match	42.10%	13.90%
Absolute improvement	42.10 pp	13.90 pp
Pretrained answer-token accuracy	6.75%	1.29%
Fine-tuned answer-token accuracy	45.60%	43.88%

Fine-tuning showed a good improvement in both models. Telugu reaches 42.10% exact match, while Nepali reaches 13.90%.

The answer-token results show additional information about the Nepali model. Its answer-token accuracy rises from 1.29% to 43.88%, even though exact-match accuracy reaches only 13.90%. Therefore, many Nepali outputs contain the expected answer token but do not match exactly.

For Nepali, reasoning difficulty also varies by task. Equality examples show almost 100% accuracy, while direct comparison, reverse comparison, transitive-2, transitive-3, and ordering remain much more difficult. Difficulty also has a large effect: easy examples reach 27.00% exact match compared with 3.68% for hard examples.

3.3 Qualitative Reasoning Examples

The following examples are taken from the actual Phase 3 evaluation outputs.

Telugu	
Successful example	**Prompt:** దినేష్ యొక్క వయస్సు శశాంక్ కంటే ఎక్కువగా ఉంది. సందీప్ యొక్క వయస్సు దినేష్ కంటే ఎక్కువగా ఉంది. దినేష్, సందీప్, శశాంక్ -- వీరిలో అత్యధిక వయస్సు ఉన్నది ఎవరు? **Gold:** సందీప్ **Fine-tuned:** సందీప్
Failure example	**Prompt:** విశాల్ యొక్క వయస్సుతో పోలిస్తే అశోక్ యొక్క వయస్సు ఎక్కువగా ఉంది. శ్రీనివాస్ యొక్క వయస్సు విశాల్ కంటే తక్కువ. శ్రీనివాస్, విశాల్, అశోక్ -- వీరిలో అత్యధిక వయస్సు ఉన్నది ఎవరు? **Gold:** అశోక్ **Fine-tuned:** విశాల్
Nepali	
Successful example	**Prompt:** विक्रम र दिपेशको तौल उस्तै छ। विक्रम र दिपेशको तौल बराबर छ? **Gold:** हो **Fine-tuned:** हो
Failure example	**Prompt:** मनोजभन्दा नविनको उचाइ धेरै देखिन्छ। मनोजभन्दा बिमलको उचाइ थोरै देखिन्छ। बिमलको उचाइ बिराजको तुलनामा बढी रहेको छ। बिराज, बिमल, नविन, मनोज मा न्यूनतम उचाइ कसको छ? **Gold:** बिराज **Fine-tuned:** बिमल

Table 12: Examples of successful and failed reasoning predictions after fine-tuning.

The successful examples show that the fine-tuned models can produce the required short answer for some comparison tasks. The failures show that the models can still make mistakes when several relations have to be combined, especially in transitive or harder ordering examples.

3.4 Reasoning/Data Mixture Experiment

Additional experiments varied the amount of synthetic reasoning data while keeping the total fine-tuning set at 20K examples.

Table 13: Reasoning-data mixture results. Values are exact-match accuracy on the same 3,000-example test set.

Reasoning / Original data	Telugu	Nepali
2K / 18K	31.07%	5.67%
5K / 15K	36.37%	10.67%
8K / 12K	37.30%	11.07%
12K / 8K	36.67%	14.60%
16K / 4K	36.47%	15.10%
20K / 0K	42.10%	13.90%

The relationship between the reasoning-data fraction and accuracy is not strictly monotonic for every intermediate mixture.

Telugu obtains its highest measured exact-match accuracy in the pure reasoning condition. Nepali reaches its highest measured mixture accuracy of 15.10% with 16K reasoning and 4K original-language examples, compared with 13.90% in the pure reasoning condition.

This experiment shows that the fine-tuning objective matters in addition to the number of reasoning examples. Adding original language-modeling examples changes the distribution of the fine-tuning objective and therefore changes the resulting task performance.

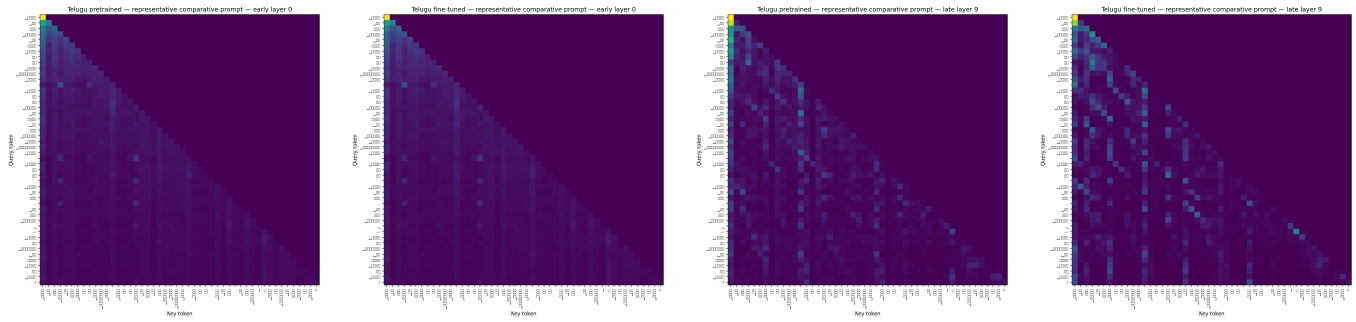
3.5 Post-Fine-tuning Attention Analysis

Attention was re-evaluated on reasoning prompts using the pretrained and fine-tuned checkpoints.

Table 14: Attention statistics before and after reasoning fine-tuning.

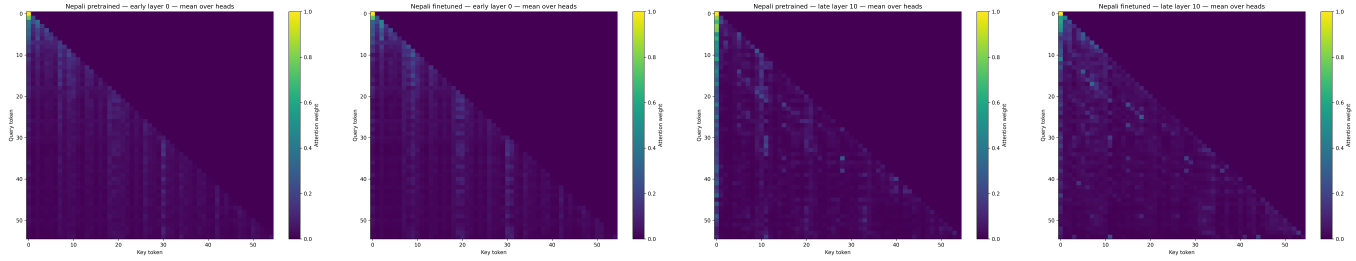
Model	State	Entropy	Distance	Δ Entropy	Δ Distance
Telugu	Pretrained	2.040	9.442	–	–
Telugu	Fine-tuned	1.894	8.891	-0.147	-0.551
Nepali	Pretrained	1.843	9.579	–	–
Nepali	Fine-tuned	1.827	8.632	-0.016	-0.947

The pretrained and fine-tuned attention maps were compared directly using the same reasoning prompts.



(a) Telugu pretrained early (b) Telugu fine-tuned early (c) Telugu pretrained late (d) Telugu fine-tuned late

Figure 5: Telugu attention before and after reasoning fine-tuning. The same early/late layers and representative reasoning setup are used for the comparison.



(a) Nepali pretrained early (b) Nepali fine-tuned early (c) Nepali pretrained late (d) Nepali fine-tuned late

Figure 6: Nepali attention before and after reasoning fine-tuning.

Fine-tuning decreases average attention entropy for both models, which means the attention becomes more concentrated on the reasoning prompts. Mean attention distance also decreases, by approximately 0.55 tokens for Telugu and 0.95 tokens for Nepali.

The change is larger for Nepali in terms of mean distance, while the entropy change is much smaller. These are aggregate changes, so they do not mean that every individual head became more local. Individual heads can still behave differently after fine-tuning.

4 Final Comparison and Discussion

Table 15: Consolidated results for the two resource tiers.

	Model H: Telugu	**Model L: Nepali**
Parameters	23.04M	25.95M
Training tokens	486.03M	500M
Vocabulary	6K	8K
Manual fraction	25.00%	18.24%
Tokenizer fertility	2.242	1.667
Characters/token	3.53	3.88
Test PPL	22.73	30.23
Test BPB	0.467	0.846
Greedy BLEU-4	8.14	3.43
Greedy repetition	54.3%	71.2%
Greedy Distinct-2	.405	.226
Reasoning exact match	42.10%	13.90%

4.1 1. How did data scale and quality differ?

The final training corpora are similar in token count, with 486.03M tokens for Telugu and 500M for Nepali. So the raw number of training tokens alone does not explain the difference in the results.

The main difference is in the availability and composition of manual data. Telugu had enough manually collected material to use a 25% manual fraction, while Nepali could use only 18.24% because the available manual corpus was smaller.

There were also differences within the Nepali corpus itself. Its OCR material has much higher tokenizer fertility than its main web sources. This suggests that the quality of the training data was not the same across all sources.

4.2 2. How did language-modeling and reasoning results compare?

Telugu obtains lower test cross-entropy, perplexity, and BPB. Its greedy generations also have lower repetition and higher diversity than Nepali.

After synthetic reasoning fine-tuning, both models improve from 0% pretrained exact match. Telugu reaches 42.10%, while Nepali reaches 13.90%. The answer-token accuracy gives some additional information for Nepali. It learns a lot of the required answer behavior, but many outputs still do not match the expected answer exactly.

So, the difference between the two models can be seen in both the language-modeling results and the synthetic reasoning results.

4.3 3. Which tokenizer and corpus factors affected Model L?

The main corpus-related limitation for Nepali was the smaller amount of manually collected data, which prevented the desired 20% manual fraction from being reached.

The tokenizer statistics also show that the Nepali corpus is quite heterogeneous. The OCR material produces much more fragmented tokenization than the main web sources. This means that some parts of the training corpus are harder to tokenize cleanly than others.

The higher test BPB of 0.846 compared with Telugu's 0.467 also shows that the difference is present beyond just the choice of subword vocabulary.

4.4 4. What evidence explains the observed differences?

Several results point to the same general differences:

- Telugu has lower held-out cross-entropy and perplexity.
- Telugu has substantially lower BPB.
- Nepali greedy generations have a higher repetition rate.
- Nepali has a smaller achievable manual-data fraction.
- Nepali contains source subsets with substantially higher tokenizer fertility, especially OCR material.
- Telugu obtains higher exact-match reasoning accuracy after fine-tuning.
- Both models show specialized local and long-range attention heads.
- Fine-tuning changes attention entropy and distance on reasoning prompts.
- Removing positional information slightly worsens Telugu intrinsic language-modeling performance.

These results suggest that corpus availability, source quality, tokenizer behavior, and resource level all have a role in the differences between the models. However, this is not a controlled causal experiment. The two languages also differ in script, linguistic structure, corpus composition, and tokenizer configuration. So the observed difference should not be attributed to just one factor.

5 Limitations

The two models cannot be treated as identical experiments differing only in resource level. Their corpora, languages, scripts, vocabulary sizes, and manually collected data differ.

Perplexity is tokenizer-dependent, making BPB more useful for cross-language intrinsic comparison.

The reasoning evaluation uses a controlled synthetic benchmark, so its accuracy should not be interpreted as a general measure of reasoning ability. Exact match is also stricter than answer-token accuracy since additional incorrect output makes an otherwise correct answer count as incorrect.

The attention analysis uses representative held-out sequences and should therefore be interpreted as evidence of attention patterns, rather than proof that a particular head always performs a specific linguistic function.

6 Conclusion

The project produced two independent approximately 25M-parameter decoder-only Transformer language models trained from scratch for Telugu and Nepali, with independent datasets, tokenizers, vocabularies, and weights.

The final Telugu model achieves test perplexity 22.73 and BPB 0.467, while the Nepali model achieves perplexity 30.23 and BPB 0.846. Generation experiments show the expected trade-off between sampling temperature, reference overlap, repetition, and diversity.

Synthetic reasoning fine-tuning improves both models. Exact-match accuracy increases from 0% to 42.10% for Telugu and from 0% to 13.90% for Nepali in the pure reasoning condition. Attention analysis shows different interaction ranges across heads, while post-fine-tuning statistics show somewhat more concentrated attention on reasoning prompts.

The positional-embedding ablation shows a small degradation in intrinsic language-modeling performance when positional information is removed, although some generation metrics move in the other direction.

Overall, similar pretraining token counts do not necessarily give the same training conditions. Manual data, source quality, tokenizer behavior, and language characteristics are relevant to the differences between the two models.